



K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

LAB FILE
DevOps and Automation Lab
(ENSP461)
Bachelor Of Technology
In
Computer Science Engineering

SUBMITTED BY:

Ritika Singh

ROLL NUMBER: 2301010097

COURSE: BTECH CSE

SECTION: B

SUBMITTED TO:

Amit Kumar Singh

School of Engineering & Technology
K. R. MANGALAM UNIVERSITY
Sohna, Haryana 122103, India

LAB 1

1. Aim

To build a foundational DevOps workflow that establishes version control, continuous integration, and automated deployment for a sample application — by setting up Git-based source control, constructing a Jenkins CI/CD pipeline, and automating the build-and-deploy process while maintaining a clean commit history and reproducible pipeline configuration.

2. Tools & Environment Used

Tool / Platform	Purpose
Windows 10 / 11	Base operating system used for this lab
Git for Windows	Distributed version control system (includes Git Bash)
GitHub	Remote repository hosting & collaboration
Visual Studio Code	Code editor used to write app code & Jenkinsfile
Jenkins (Windows installer)	Automation server used to build the CI/CD pipeline

3. Theory

3.1 What is DevOps?

DevOps simply means Development + Operations working together as one team instead of working as two separate teams.

DevOps culture helps bridge this gap by using automation, continuous testing, and continuous delivery. This allows software to reach production faster, more reliably, and with fewer errors.

3.2 Version Control & Git

A **Version Control System (VCS)** is a tool that keeps track of every change made to the code over time. It records **who made the change, when it was made, and what was changed**. If something goes wrong, developers can easily go back to an earlier version of the code.

Git is one of the most widely used **distributed version control systems (DVCS)**. The word **distributed** means that every developer has a complete copy of the repository on their local computer. Therefore, developers do not have to depend completely on a single central server to manage the code.

3.3 Continuous Integration (CI)

Continuous Integration (CI) is a software development practice in which developers frequently merge their code into a shared repository, often multiple times a day. After every merge, the code is automatically built and tested.

- **Benefit 1:** Bugs are detected early when the changes are small and easier to fix.
- **Benefit 2:** Manual testing effort is reduced because automation handles the testing process.
- **Benefit 3:** The team always has a working and integrated version of the code to work with.

3.4 Continuous Deployment / Delivery (CD)

Continuous Delivery means that the code is always kept in a release-ready state. **Continuous Deployment** takes this one step further — after every

successful build and test, the application is automatically deployed to production without requiring manual approval.

3.5 Jenkins

Jenkins is an open-source automation server used to create and manage **CI/CD pipelines**. It detects changes in source code, such as changes made in a GitHub repository, and automatically performs tasks like **building, testing, and deploying** the application. It also reports the results of these tasks.

A Jenkins pipeline is usually defined in a **Jenkinsfile**. It is a text file that contains all the stages of the pipeline, such as **Build, Test, and Deploy**, written in code. This makes the pipeline **version-controlled, consistent, and reproducible**.

3.6 CI/CD Pipeline — Visual Flow

Stage	What Happens
1. Code Commit	Developer commits and pushes the code to the Git repository.
2. Trigger	Jenkins detects the new push on GitHub using a webhook or polling.
3. Build	Jenkins builds and compiles the application automatically.
4. Test	Automated tests are executed to check the application. Any failures are detected immediately.
5. Deploy	If the build and tests are successful, Jenkins deploys the application to the target environment.

4. Installation & Setup Procedure

4.1 Install Git (Windows)

1. Download the **Git for Windows installer** from the official Git website.
2. Run the installer and follow the setup instructions. You can keep the **default options**, which will install **Git Bash** and **Git CMD** along with Git.
3. Verify the installation by opening **Git Bash** or **PowerShell** and running:

`git --version`

4.2 Configure Git Identity

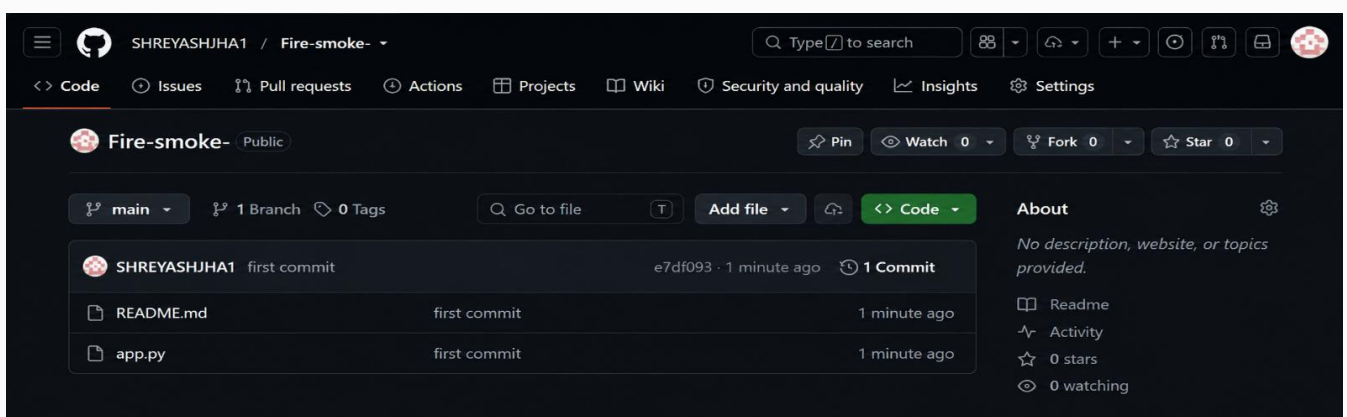
`git config --global user.name "Shreyash Jha"`

`git config --global user.email "your-email@example.com"`

These commands can be executed in both **Git Bash** and **PowerShell** in the same way.

4.3 Create a GitHub Repository

1. GitHub par login karo → New Repository → naam do (e.g. devops-lab1-cicd).
2. Keep the repository Public and initialize it with a README.md file.
3. Repo ka URL copy karo: <https://github.com/SHREYASHJHA112/Fire-smoke-.git>



4.4 Clone & Set Up Local Repository

```
git clone https://github.com/SHREYASHJHA112/Fire-smoke-.git
```

```
cd Fire-smoke-
```

```
echo "print('First Commit')" > app.py
```

```
git add .
```

```
git commit -m "Initial commit: add sample app"
```

```
git push origin main
```

```
PS C:\fire-smoke-detection> git init
Reinitialized existing Git repository in C:/fire-smoke-detection/.git/
PS C:\fire-smoke-detection> git add README.md app.py
warning: in the working copy of 'app.py', LF will be replaced by CRLF the next time Git touches it
```

```
PS C:\fire-smoke-detection> git commit -m "first commit"
Author identity unknown

*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

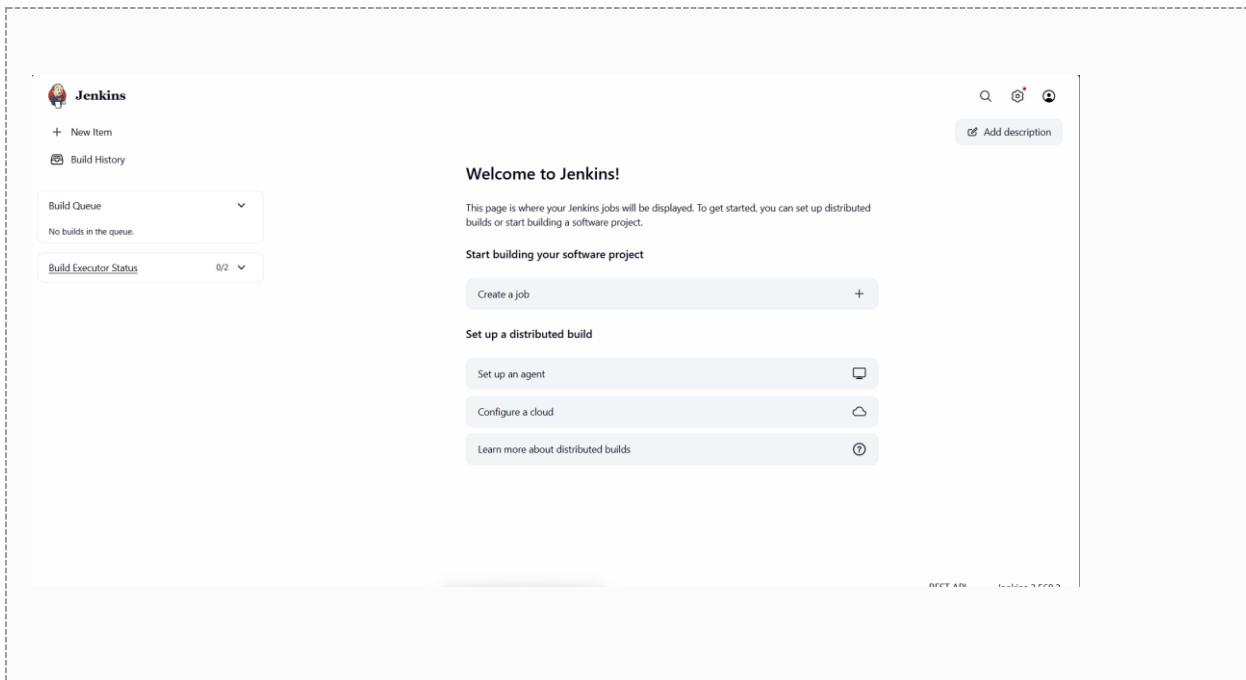
fatal: unable to auto-detect email address (got 'ag683@SHREYASHJHA.(none)')
PS C:\fire-smoke-detection> git config --global user.email "ag6830221@gmail.com"
>> git config --global user.name "Shreyash Jha"
PS C:\fire-smoke-detection> git commit -m "first commit"
[main (root-commit) e7df093] first commit
 2 files changed, 797 insertions(+)
 create mode 100644 README.md
 create mode 100644 app.py
```

```
PS C:\fire-smoke-detection> git branch -M main
PS C:\fire-smoke-detection> git remote add origin https://github.com/ADITYAGUPTA112/Fire-smoke-.git
error: remote origin already exists.
PS C:\fire-smoke-detection> git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 20 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 9.77 KiB | 9.77 MiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/ADITYAGUPTA112/Fire-smoke-.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

4.5 Install Jenkins on Windows

1. Jenkins Installation on Windows

2. **Install Java first** — Jenkins requires **JDK 17**. Download and install **Eclipse Temurin 17 (LTS)** from Adoptium.
3. **Verify Java installation** in Command Prompt:
4. `java -version`
5. **Download the Jenkins Windows installer (.msi)** from the Jenkins official website and select the **Windows** section.
6. **Run the jenkins.msi installer.** The setup wizard will ask for the installation path and port. Keep the default settings, including **port 8080**.
7. The installer automatically registers Jenkins as a **Windows Service** and starts it.
8. Open a web browser and go to:
`http://localhost:8080`
9. The **Unlock Jenkins** screen will appear.
10. The initial administrator password can be found in the following file:
11. `C:\Program Files\Jenkins\secrets\initialAdminPassword`
12. Open the file using Notepad and copy the password.
13. Paste the password into the **Unlock Jenkins** screen. Select **Install suggested plugins** and then create an **admin user**.
14. After completing these steps, Jenkins will be successfully installed and ready to use on Windows.



4.6 Create a Jenkins Pipeline Job

1. Open the **Jenkins Dashboard** and click **New Item**.
2. Enter a name for the pipeline, select **Pipeline**, and click **OK**.
3. In the **Pipeline** section, select **Pipeline script from SCM**.
4. Set **SCM** to **Git** and enter your **GitHub repository URL**.
5. Specify the branch, for example:
6. `*/main`
7. Set **Script Path** to:
8. `Jenkinsfile`
9. Click **Save** and then click **Build Now** to start the pipeline.
10. Jenkins will fetch the code from the GitHub repository and execute the stages defined in the **Jenkinsfile**.

General

Enabled 

Description

[Plain text](#) [Preview](#)

☒ Discard old builds [?](#)

Strategy

Long Rotation

Days to keep builds

If not empty, build records are only kept up to this number of days

Max # of builds to keep

If not empty, only up to this number of build records are kept

10

[Advanced](#) 

☒ Do not allow concurrent builds

☐ Abort previous builds [?](#)

☐ Do not allow the pipeline to resume if the controller restarts

☐ GitHub project

☐ Pipeline speed/durability override [?](#)

☐ Preserve stages from completed builds [?](#)

☒ This project is parameterized [?](#)

 Boolean Parameter [?](#)

Name [?](#)

RUN_FULL_TESTS

☒ Set by Default [?](#)

Description [?](#)

Run full unit and model integrity test suite

[Plain text](#) [Preview](#)

 Boolean Parameter [?](#)

Name [?](#)

PACKAGE_ARTIFACTS

☒ Set by Default [?](#)

Description [?](#)

Package application and model artifacts upon build completion

[Plain text](#) [Preview](#)

 Choice Parameter [?](#)

Name [?](#)

DEPLOY_ENV

Options [?](#)

staging
production
none

Description [?](#)

Target deployment environment

[Plain text](#) [Preview](#)

[+ Add Parameter](#)

☐ Throttle builds [?](#)

Triggers

Set up automated actions that start your build based on specific events, file code changes or scheduled times.

☐ Build after other projects are built [?](#)

☐ Build periodically [?](#)

☐ GitHub hook trigger for GITSCM polling [?](#)

☐ Poll SCM [?](#)

☐ Trigger builds remotely (e.g., from scripts) [?](#)

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script from SCM

SCM [?](#)

Git

Repositories [?](#)

Repository URL [?](#)

https://github.com/ADITYAGUPTA12/free-smoke-git

Credentials [?](#)

- none -

[+ Add](#)

[Advanced](#) 

[+ Add Repository](#)

Branches to build [?](#)

Branch Specifier (blank for 'any') [?](#)

/main



Free-Smoke-Detection-Cl

Configure

Configure

 General

 Triggers

 Pipeline

 Advanced

Repository browser [?](#)

(Auto)

Additional Behaviours

[+ Add](#)

Script Path [?](#)

Jenkinsfile

☒ Lightweight checkout [?](#)

[Pipeline Syntax](#)

Advanced

[Advanced](#) 

[Save](#)

[Apply](#)

4.7 Jenkinsfile

The file should be saved in the **root directory of the repository** with the name **Jenkinsfile**. It should have **no file extension** such as .txt or .groovy.

For **Windows agents**, use **bat** instead of **sh** for executing shell commands.

```
pipeline {
    agent any

    options {
        timeout(time: 30, unit: 'MINUTES')
        buildDiscarder(logRotator(numToKeepStr: '10'))
        disableConcurrentBuilds()
    }

    environment {
        PYTHON_APP = 'app.py'
        MODEL_PATH = 'fire_smoke_model.pkl'
        SCALER_PATH = 'scaler.pkl'
        DATASET_PATH = 'smoke_detection_iot.csv'
        VENV_DIR = '.venv'
    }

    parameters {
        booleanParam(name: 'RUN_FULL_TESTS', defaultValue: true,
description: 'Run full unit and model integrity test suite')

        booleanParam(name: 'PACKAGE_ARTIFACTS', defaultValue: true,
description: 'Package application and model artifacts upon build
completion')
```

```
    choice(name: 'DEPLOY_ENV', choices: ['staging', 'production', 'none'],
description: 'Target deployment environment')
}
```

```
stages {
  stage('Checkout') {
    steps {
      echo '=== Stage 1: Checkout Source Code ==='
      checkout scm
      script {
        echo "Building Commit: ${env.GIT_COMMIT ?: 'Local Workspace'}"
        echo "Branch: ${env.GIT_BRANCH ?: 'main'}"
      }
    }
  }
}
```

```
stage('Setup Environment & Dependencies') {
  steps {
    echo '=== Stage 2: Preparing Python Environment ==='
    script {
      if (isUnix()) {
        sh '''
          python3 -m venv ${VENV_DIR} || virtualenv ${VENV_DIR}
          . ${VENV_DIR}/bin/activate
          pip install --upgrade pip
          pip install -r requirements.txt
          '''
      }
    }
  }
}
```

```

    } else {
        bat ""

        @echo off

        set "PY_EXE=python"

        where python >nul 2>nul

        if %errorlevel% neq 0 (
            where py >nul 2>nul

            if %errorlevel% equ 0 (
                set "PY_EXE=py"
            ) else if exist
"C:\Users\lag683\AppData\Local\Programs\Python\Python311\python
.exe" (
                set
"PY_EXE=C:\Users\lag683\AppData\Local\Programs\Python\Python311
\python.exe"
            ) else if exist "C:\Windows\py.exe" (
                set "PY_EXE=C:\Windows\py.exe"
            )
        )

        echo Using Python executable: %PY_EXE%

        %PY_EXE% -m venv %VENV_DIR%

        call %VENV_DIR%\Scripts\activate.bat

        %VENV_DIR%\Scripts\python.exe -m pip install --upgrade pip

        %VENV_DIR%\Scripts\pip.exe install -r requirements.txt

        ""
    }
}

```

```
}  
}
```

```
stage('Code Quality & Syntax Check') {  
    steps {  
        echo '=== Stage 3: Running Code Quality Checks ==='  
        script {  
            if (isUnix()) {  
                sh '''  
                    . ${VENV_DIR}/bin/activate  
                    python3 -m py_compile ${PYTHON_APP} DA.PY test_model.py  
                '''  
            } else {  
                bat '''  
                    call %VENV_DIR%\Scripts\activate.bat  
                    %VENV_DIR%\Scripts\python.exe -m py_compile  
%PYTHON_APP% DA.PY test_model.py  
                '''  
            }  
        }  
    }  
}
```

```
stage('Model & Artifact Integrity Verification') {  
    steps {  
        echo '=== Stage 4: Verifying Machine Learning Artifacts ==='  
        script {
```

```

        if (isUnix()) {
            sh ""
                . ${VENV_DIR}/bin/activate
            python3 -c "
import os, joblib
assert os.path.exists('${MODEL_PATH}'), 'Model artifact missing'
assert os.path.exists('${SCALER_PATH}'), 'Scaler artifact missing'
model = joblib.load('${MODEL_PATH}')
scaler = joblib.load('${SCALER_PATH}')
print('SUCCESS: Model and Scaler artifacts loaded successfully!')
"
            ""
        } else {
            bat ""
                call %VENV_DIR%\Scripts\activate.bat
                %VENV_DIR%\Scripts\python.exe -c "import os, joblib; assert
os.path.exists('%MODEL_PATH%'); assert
os.path.exists('%SCALER_PATH%'); model = joblib.load('%MODEL_PATH%');
scaler = joblib.load('%SCALER_PATH%'); print('SUCCESS: Model and Scaler
artifacts loaded successfully!')"
            ""
        }
    }
}

stage('Unit & Model Tests') {

```

```

when {
    expression { return params.RUN_FULL_TESTS }
}

steps {
    echo '=== Stage 5: Executing Test Suite ==='
    script {
        if (isUnix()) {
            sh '''
                . ${VENV_DIR}/bin/activate
                python3 -m unittest test_model.py -v
            '''
        } else {
            bat '''
                call %VENV_DIR%\Scripts\activate.bat
                %VENV_DIR%\Scripts\python.exe -m unittest test_model.py -
v
            '''
        }
    }
}

stage('Package Release Bundle') {
    when {
        expression { return params.PACKAGE_ARTIFACTS }
    }
    steps {

```

```
echo '=== Stage 6: Packaging Release Artifacts ==='

script {

    def releaseFileName = "fire-smoke-detection-
b${env.BUILD_NUMBER ?: '1'}.zip"

    if (isUnix()) {

        sh "zip -r ${releaseFileName} app.py DA.PY
fire_smoke_model.pkl scaler.pkl requirements.txt README.md"

    } else {

        powershell "Compress-Archive -Path app.py, DA.PY,
fire_smoke_model.pkl, scaler.pkl, requirements.txt, README.md -
DestinationPath ${releaseFileName} -Force"

    }

    echo "Created release artifact package: ${releaseFileName}"

}

}

stage('Deploy Application') {

    when {

        expression { return params.DEPLOY_ENV != 'none' }

    }

    steps {

        echo "=== Stage 7: Triggering Deployment to
[${params.DEPLOY_ENV}] ==="

        script {

            echo "Deploying Industrial Fire & Smoke Radar System to
${params.DEPLOY_ENV} cluster..."

        }

    }

}
```

```
        // Add remote deployment execution script / helm / ssh / docker
        commands here
```

```
    }
```

```
  }
```

```
}
```

```
}
```

```
post{
```

```
  always {
```

```
    echo '=== Pipeline Execution Completed ==='
```

```
    cleanWs deleteDirs: true, notFailBuild: true, patterns: [[pattern: '*.pyc',
type: 'INCLUDE']]
```

```
  }
```

```
  success {
```

```
    echo '✅ SUCCESS: Jenkins Pipeline completed successfully!'
```

```
  }
```

```
  failure {
```

```
    echo '❌ FAILURE: Jenkins Pipeline failed during execution.'
```

```
  }
```

```
}
```

```
}
```



Jenkins

/ Fire-Smoke-Detection-CI

#6



Status



#6 (10 Aug 2026, 11:56:14 pm)

Add description

Keep this build forever

Changes



Started by user [Aditya Gupta](#)

Started 4 min 8 sec ago
Took 4 min 0 sec

Console Output

Delete build '#6'

Parameters

Git Build Data



Revision: 2d5390896ca20da24b58fdce0a93467b162b0e92

Repository: <https://github.com/ADITYAGUPTA112/Fire-smoke-git>

• origin/main

Pipeline Overview



Fix Windows Python executable resolution in Jenkinsfile 2d53908

Aditya Gupta at 11:55 pm 10/08/26

Edit build information

Restart from Stage

Replay

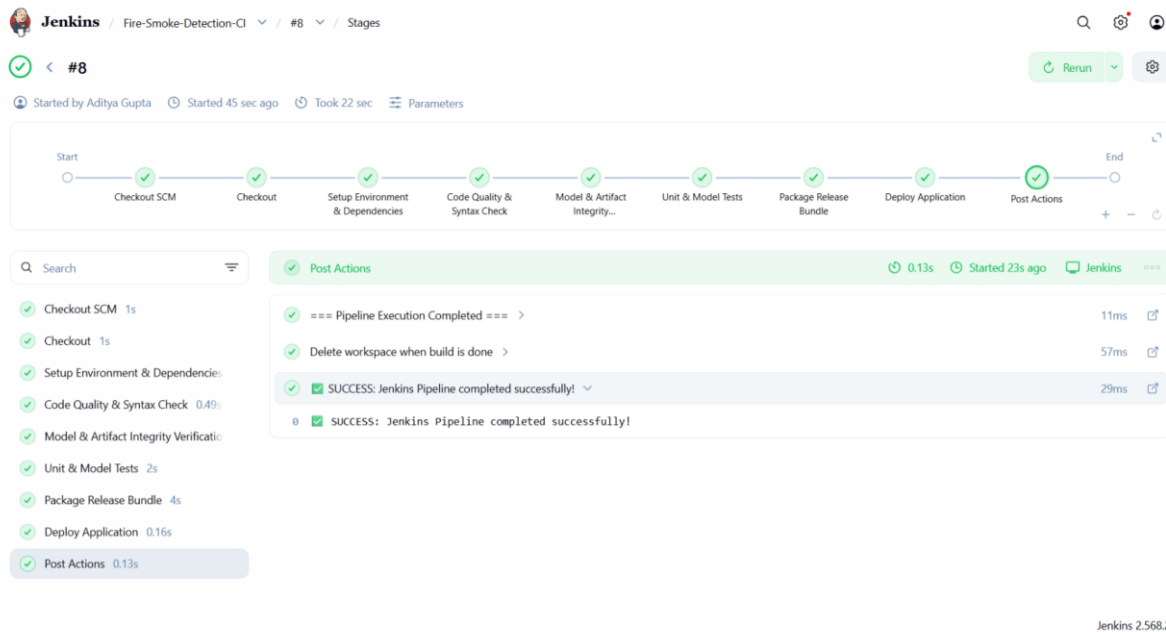
Pipeline Steps

Workspaces

Previous Build

REST API

Jenkins 2.568.2



5. Observations

- **Git was successfully installed**, and the local repository was successfully connected to the GitHub repository.
- Each commit was made with a **meaningful commit message**, keeping the commit history clean and easy to track.
- After installing Jenkins, the code was automatically fetched from the **GitHub repository** whenever a new push was made.
- The **Build → Test → Deploy** stages of the pipeline executed successfully and were shown with **green check marks** in the Jenkins Stage View.
- This process eliminated the need for **manual deployment** and made the entire workflow **automated, consistent, and reproducible**.

6. Conclusion

In this lab, we successfully set up a **foundational DevOps workflow**. We used **Git and GitHub** for version control and **Jenkins** to create a working **CI/CD pipeline**. The pipeline automatically **builds, tests, and deploys** the application whenever new code is committed.

This lab helped us understand how **DevOps automation** reduces the gap between development and operations. It also makes the software delivery process **faster, more reliable, consistent, and reproducible**.